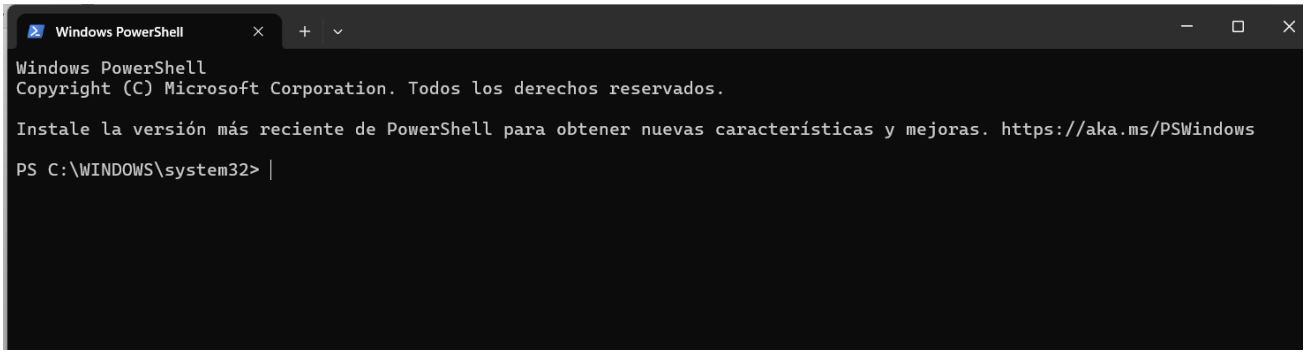
EJERCICIO. BASES DE DATOS

El objetivo de este ejercicio es aprender a usar Bases de Datos en entorno SQL Server

En este módulo aprenderemos temas relacionados con las bases de Datos en general y SQL Server en particular.

En este ejercicio nos centramos en la Consola de SQL SERVER

Abrimos Power Shell en windows



Nos conectamos a algún directorio donde tengamos una carpeta para almacenar el código que generemos

Ejemplo:

D:

$> mkdir proyectos\BaseDatos

$> cd Proyectos\Basedatos

Crear una Base de Datos

Conectarse a una Base de Datos creada

Nuestra Base de Datos se llama **bdprueba01**

Está en el servidor Local

Para conectarnos a ella, haremos:

```
Windows PowerShell - SQLCM
PS D:\proyectos\BaseDatos>
PS D:\proyectos\BaseDatos>
PS D:\proyectos\BaseDatos>
PS D:\proyectos\BaseDatos> sqlcmd -S "(localdb)\MSSQLLocalDB" -d bdprueba01 -E
1>
```

Como sale el 1> , significa que ya está esperando comandos

Los comandos los terminamos con punto y coma “;”

y ponemos el comando **GO**, para que SQL Server ejecute el comando de Base de Datos

Por ejemplo:

Para saber como se llama la BBDD a la que estamos conectados

```
1> SELECT DB_NAME() AS BaseDeDatosActual;
2> GO
```

Nos responderá…

```
BaseDeDatosActual
----------------------------------------------
bdprueba01
```

Sabemos que tenemos una Tabla de Provincias creada

Vamos a acceder a ella.

```
1> select * from dbo.provincias;
2> GO
```

```
idprovincia provincia

----------- --------------------

          1 sevilla

          2 cadiz

          3 huelva

          4 malaga

          5 granada

          6 almeria

          7 jaen

          8 cordoba

(8 rows affected)
```

Recuerda: dbo significa el nombre del Schema de Base de Datos. Es como el contenedor donde están tus tablas.

Si queremos saber las tablas que tenemos

```
SELECT name AS tabla
FROM sys.tables
ORDER BY name;
GO
```

Para salir

Tecleamos exit y volvemos a salir al Powershell

```
1> exit
PS D:\proyectos\BaseDatos>
PS D:\proyectos\BaseDatos>
PS D:\proyectos\BaseDatos>
```

Para lanzar un script .sql

Si tu script se llama por ejemplo MiScript.sql:

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d bdprueba01 -E -i "ruta\MiScript.sql"
```

- -S servidor/instancia
- -d base de datos
- -E autenticación Windows
- -i input file (el script)

Guardar el resultado en un archivo si quieres.

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d bdprueba01 -E -i "ruta\script.sql" -o “ruta\salida.txt"
```

CONCEPTO IMPORTANTE

Una Base de Datos puede tener varios Schemas

Cada Schema tiene objetos (Tablas, vistas, procedimientos, funciones, índices, etc)
Una **base de datos** puede tener **varios schemas**
Un **schema NO contiene varias bases de datos** (un schema "vive" dentro de **una** base de datos).

Como se nombra los accesos a cada objeto

BASE-DE-DATOS . SCHEMA . TABLA u Objeto

Cómo se relacionan

- **Servidor SQL Server (instancia)** → tiene **varias bases de datos**
- **Cada base de datos** → puede tener **muchos schemas**
- **Cada schema** → contiene objetos **de esa base de datos** (tablas, vistas, procedimientos, funciones…)

Ejemplo claro

En la base de datos bdprueba01 puedes tener:

- dbo.provincias
- ventas.provincias
- rrhh.empleados

Son objetos en **la misma base de datos**, pero en **schemas distintos**.

En nuestro ejemplo

bdprueba01.dbo.provincias

¿Cómo CREO UN SCHEMA NUEVO?

```
USE bdprueba01;
GO
CREATE SCHEMA ventas;
GO
```

¿COMO SABEMOS LOS SCHEMAS QUE TENEMOS?

```
SELECT name FROM sys.schemas ORDER BY name;
GO
```

Si queremos saber las tablas que tenemos

```
SELECT name AS tabla
FROM sys.tables
ORDER BY name;
GO
```

Para salir

Tecleamos exit y volvemos a salir al Powershell

4) Mover una tabla existente a otro schema (opcional)

Si ya tienes dbo.provincias y quieres pasarla a ventas:

```
ALTER SCHEMA ventas TRANSFER dbo.provincias;
```

PERMISOS SOBRE UN SCHEMA. Se verá más adelante

Por ejemplo, permitir que un usuario cree/lea/escriba dentro de ventas:

GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::ventas TO [TuUsuario];

GO

TEMAS DE GESTION DE USUARIOS

Se verá más adelante

Para saber **con qué usuario estás conectado** en SQL Server (desde sqlcmd, SSMS o Azure Data Studio), ejecuta cualquiera de estas consultas:

Usuario de inicio de sesión (login) y usuario dentro de la BD

```
SELECT
  SUSER_SNAME()  AS LoginWindowsOSQL,
  USER_NAME()    AS UsuarioEnLaBD,
  ORIGINAL_LOGIN() AS LoginOriginal;
GO
```

También muy útil: quién soy y en qué servidor/BD estoy

```
SELECT
  @@SERVERNAME AS Servidor,
  DB_NAME() AS BaseDeDatos,
  SYSTEM_USER  AS SystemUser,
  CURRENT_USER AS CurrentUser;
GO
```

RESUMEN:

- SUSER_SNAME() / SYSTEM_USER → tu **login** (normalmente tu usuario Windows si usas -E)
- USER_NAME() / CURRENT_USER → el **usuario mapeado dentro de esa base de datos** (a menudo dbo)

```
1> SELECT
2>   SUSER_SNAME()  AS Login,
3>   USER_NAME()    AS UsuarioEnBD,
4>   SYSTEM_USER    AS SystemUser,
5>   CURRENT_USER   AS CurrentUser;
6> GO
Login              UsuarioEnBD       SystemUser        CurrentUser
PORTATIL\manel     dbo               PORTATIL\manel    dbo
```

INSTANCIA DE LA BASE DE DATOS

Para saber si la instancia de BBDD está parada

```
PS C:\WINDOWS\system32> sqllocaldb i
MSSQLLocalDB
PS C:\WINDOWS\system32> sqllocaldb start MSSQLLocalDB
Instancia 'MSSQLLocalDB' de LocalDB iniciada.
PS C:\WINDOWS\system32> sqllocaldb stop MSSQLLocalDB
Instancia 'MSSQLLocalDB' de LocalDB detenida.
PS C:\WINDOWS\system32> sqllocaldb start MSSQLLocalDB
Instancia 'MSSQLLocalDB' de LocalDB iniciada.
PS C:\WINDOWS\system32>
```

Si usas SQL Server como servicio (SQL Server Express/Developer/Standard)

Aquí se controla como **servicio de Windows.**

Opción 1: desde consola (PowerShell como admin)

Primero, ver servicios SQL:

```
Get-Service *SQL* | Select Name, Status
```

Arrancar (ejemplo de instancia SQLEXPRESS):

```
Start-Service "MSSQL$SQLEXPRESS"
```

Parar:

```
Stop-Service "MSSQL$SQLEXPRESS"
```

Para la instancia “por defecto” (sin nombre):

```
Start-Service "MSSQLSERVER"

Stop-Service "MSSQLSERVER"
```

Esto lo haces desde POWERSHELL